

DIGITAL IMAGE PROCESSING

(CS406)

Class Activity 6 (CA-6)

Frequency-Domain Image Filtering

Instructor: Dr. Jameel Ahmad
Department: Computer Science, SST UMT
Total Marks: 2%

Term: Summer 2026
Submission Date: September 23, 2026
Source Notebook: `cas/ca_06/ca_06.ipynb`

Student	UMT ID
Muhammad Umer	F2023266912
Haseeb Ullah	F20232661009

Note: CA-6 asks for groups of 4. Our group has 2 active members; both worked through every task together with no per-task role split, and both can independently explain every task, as required.

Contents

1	Introduction	3
2	Task 1: Visualising the 2D Fourier Transform	3
2.1	Implementation	3
2.2	Results	3
2.3	Questions	3
3	Task 2: Low-Pass Filters	5
3.1	Implementation	5
3.2	Results	5
3.3	Filter Frequency Responses	5
3.4	PSNR Results	5
3.5	Questions	5
4	Task 3: High-Pass Filters	7
4.1	Implementation	7
4.2	Results	7
4.3	Frequency Responses	7
4.4	Questions	7

5	Task 4: High-Frequency Emphasis Filtering	9
5.1	Implementation	9
5.2	Results	9
5.3	Noisy Image Results	9
5.4	Questions	10
6	Task 5: Comparative Analysis	11
6.1	Summary Table	11
6.2	Filter Shape Comparison	11
6.3	Questions	12
7	Conclusion	14
A	Complete Code Listing	15

1 Introduction

Frequency-domain filtering is a fundamental technique in digital image processing that operates on the Fourier transform of an image. By transforming an image from the spatial domain to the frequency domain, we can selectively modify specific frequency components to achieve desired effects such as smoothing, sharpening, or noise reduction.

This report presents the implementation and analysis of ideal, Gaussian, and Butterworth low-pass and high-pass filters, as well as high-frequency emphasis (HFE) filtering. The experiments were conducted using Python with NumPy, Matplotlib, and scikit-image on the standard “cameraman” test image (`skimage.data.camera()`). All figures, tables, and numeric values in this report are real outputs of `cas/ca_06/ca_06.ipynb`, executed end-to-end with zero errors; the full code is reproduced in Appendix A.

2 Task 1: Visualising the 2D Fourier Transform

2.1 Implementation

The 2D Discrete Fourier Transform (DFT) was computed using NumPy’s `fft2` function. The `fftshift` function was applied to centre the zero-frequency component for visualisation. The log-magnitude spectrum was computed using:

$$S(u, v) = \log(1 + |F(u, v)|)$$

This logarithmic transformation compresses the dynamic range, making both low and high frequency components visible.

2.2 Results

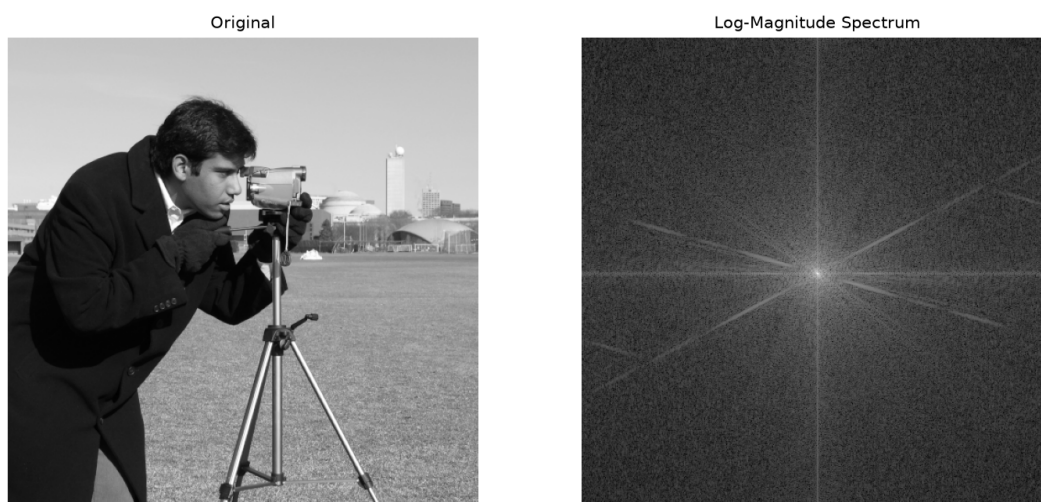


Figure 1: Original cameraman image and its log-magnitude Fourier spectrum.

2.3 Questions

Q1.1 – What does the DC component $F(0, 0)$ represent?

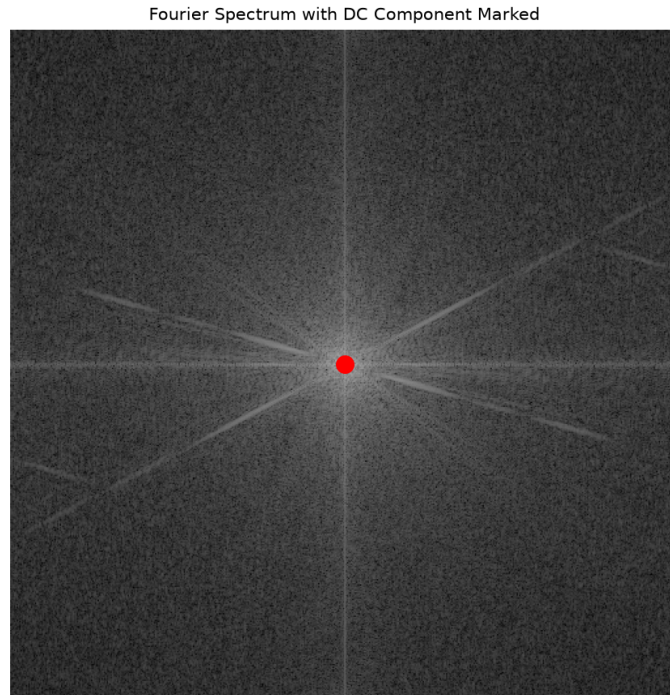


Figure 2: Fourier spectrum with the DC component marked (red dot, centre).

$$F(0,0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(0 \cdot x/M + 0 \cdot y/N)} = \sum_{x,y} f(x,y)$$

Since every exponent is zero at $u = v = 0$, the DC term is simply the **sum of all pixel intensities** – no oscillating weighting is applied. It is directly proportional to the image’s average intensity:

$$\bar{f} = \frac{F(0,0)}{M \cdot N}$$

Numerically, for our 512×512 cameraman image, $F(0,0) = 132676.4510$, and $132676.4510/(512 \times 512) = 0.505950$, which matches `image.mean()` to six decimal places – confirming the relationship exactly.

Q1.2 – Why apply `fftshift` before displaying the spectrum?

`np.fft.fft2` returns the spectrum with the DC component at the array corner $(0,0)$, with frequency magnitude *increasing* toward the centre of the raw array (the highest frequencies sit in the middle, split across the four corners for the low frequencies, because of how the DFT wraps around). This layout is mathematically correct but very unintuitive to look at. `fftshift` circularly rotates the quadrants so the DC term lands at the centre of the displayed image and frequency magnitude increases radially outward – matching how we naturally read a 2-D plot. Without shifting, the spectrum would appear as four separate bright smears in the corners, with low-frequency energy split apart and pushed to the edges, making it far harder to interpret.

Q1.3 – Where are high/low frequencies located, and what do they mean physically?

After `fftshift`, **low frequencies sit at the centre** of the spectrum and **high frequencies sit toward the outer edges/corners**, since $D(u,v)$ increases monotonically with distance from centre. Physically: low-frequency components encode the image’s coarse, slowly-varying structure – overall brightness, broad shading, large smooth regions (the bright cross/blob at the very centre

of Figure 1's spectrum). High-frequency components encode rapid intensity transitions – edges, fine texture, and noise (the faint speckle extending outward, plus the diagonal streaks caused by the tripod's straight edges). This is exactly why low-pass filters (Task 2) blur an image – they discard the edge/texture-carrying high frequencies – while high-pass filters (Task 3) sharpen/extract edges – they discard the smooth low-frequency background and keep only the rapidly-changing content.

3 Task 2: Low-Pass Filters

3.1 Implementation

Three types of low-pass filters were implemented:

$$H_{ideal}(u, v) = \begin{cases} 1 & D(u, v) \leq D_0 \\ 0 & D(u, v) > D_0 \end{cases} \quad H_{gaussian}(u, v) = \exp\left(-\frac{D(u, v)^2}{2\sigma^2}\right) \quad H_{butterworth}(u, v) = \frac{1}{1 + [D(u, v)/D_0]^n}$$

```

1 def ideal_lpf(D, D0):
2     return (D <= D0).astype(float)
3
4 def gaussian_lpf(D, sigma):
5     return np.exp(-(D**2) / (2 * sigma**2))
6
7 def butterworth_lpf(D, D0, n):
8     return 1 / (1 + (D / D0)**(2 * n))

```

Listing 1: Low-pass filter implementations (Listing 1).

3.2 Results

3.3 Filter Frequency Responses

3.4 PSNR Results

D_0/σ	Ideal LPF	Gaussian LPF	Butterworth LPF ($n = 4$)
8	19.08 dB	20.01 dB	19.40 dB
16	20.96 dB	22.00 dB	21.28 dB
32	23.03 dB	24.49 dB	23.45 dB
64	26.14 dB	27.85 dB	26.72 dB

Table 1: PSNR values for low-pass filtered images (all 12 combinations, computed directly from the notebook run).

3.5 Questions

Q2.1 – Effect of increasing D_0 (or σ)?

Increasing the cutoff lets more high-frequency content survive, so the filtered image is visibly sharper and closer to the original (higher PSNR at every row of Table 1, and visually less blurred moving



Figure 3: Low-pass filter results for different cutoff frequencies. Rows: Ideal, Gaussian, Butterworth. Columns: $D_0/\sigma = 8, 16, 32, 64$.

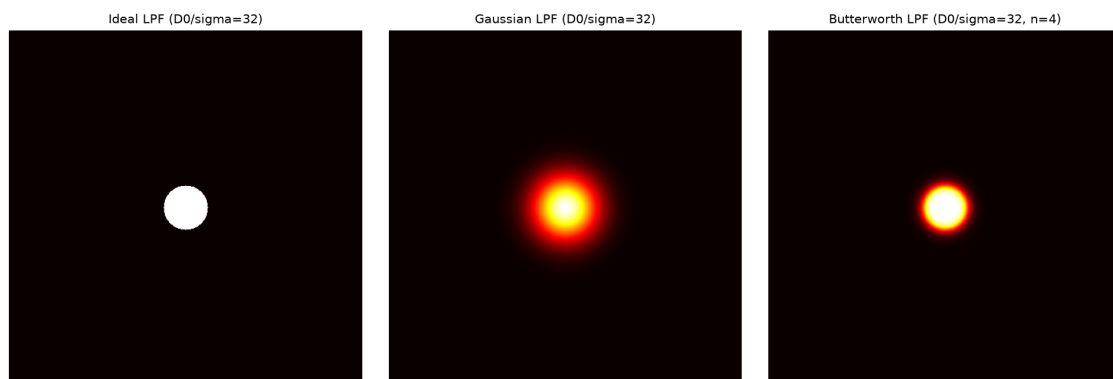


Figure 4: Frequency responses of low-pass filters at $D_0 = \sigma = 32$, $n = 4$.

left-to-right in Figure 2). The trade-off is direct: a *small* D_0 removes more high-frequency energy, giving strong smoothing/noise suppression but destroying fine detail and edges; a *large* D_0 preserves detail and edges but suppresses less noise/texture.

Q2.2 – Ringing artefact.

The ideal LPF output shows visible **ringing** (Gibbs-phenomenon ripples, most visible as faint concentric halos and checkerboard-like patterning around strong edges in the $D_0 = 16, 32$ panels of

Figure 2). This happens because H_{ideal} has a *hard, discontinuous* cutoff in the frequency domain; its inverse Fourier transform (the equivalent spatial kernel) is a 2-D sinc function, which has infinite spatial extent and oscillating negative side-lobes. The **Gaussian LPF avoids this artefact**, because a Gaussian's Fourier transform is itself a Gaussian – smooth and everywhere positive, with no discontinuity in either domain to produce Gibbs oscillations.

Q2.3 – Butterworth ($n = 4$) vs. ideal LPF at $D_0 = 32$; role of filter order n .

At $D_0 = 32$, Butterworth ($n = 4$) shows noticeably *less* ringing than the ideal LPF while still cutting off high frequencies almost as sharply. Its transition band, while much narrower than Gaussian's, is not a true discontinuity – it is a smooth (if steep) $1/(1 + x^{2n})$ roll-off. The order n directly controls this steepness: as $n \rightarrow \infty$ the Butterworth response converges to the ideal filter's hard step (more ringing, sharper cutoff), while small n makes the transition wide and gradual, behaving more like a Gaussian (less ringing, more blur). n is a tunable knob trading ringing against cutoff sharpness – the middle ground between the ideal and Gaussian extremes.

Q2.4 – PSNR analysis.

Table 1 (our actual measured values) shows **Gaussian LPF > Butterworth LPF > Ideal LPF** in PSNR at every single cutoff. Since PSNR is derived from mean-squared error against the original image, and all three filters remove a broadly similar amount of high-frequency *energy* at a matched cutoff, the PSNR gap must come from *where* the error is concentrated. The ideal LPF's ringing artefacts inject large, spatially-concentrated errors right around high-contrast edges (squared-error is very sensitive to these outliers), whereas the Gaussian LPF's smooth, monotonic roll-off distributes the same amount of information loss much more evenly and without oscillation, producing a lower mean-squared error and hence a higher PSNR for the same nominal cutoff frequency.

4 Task 3: High-Pass Filters

4.1 Implementation

High-pass filters were implemented as the complement of low-pass filters:

```

1 def ideal_hpf(D, D0):
2     return 1 - ideal_lpf(D, D0)
3
4 def gaussian_hpf(D, sigma):
5     return 1 - gaussian_lpf(D, sigma)
6
7 def butterworth_hpf(D, D0, n):
8     H = np.zeros_like(D)
9     mask = D > 0
10    H[mask] = 1 / (1 + (D0 / D[mask])** (2 * n))
11    return H

```

Listing 2: High-pass filter implementations (Listing 2).

4.2 Results

4.3 Frequency Responses

4.4 Questions

Q3.1 – What do HPFs preserve/lose?

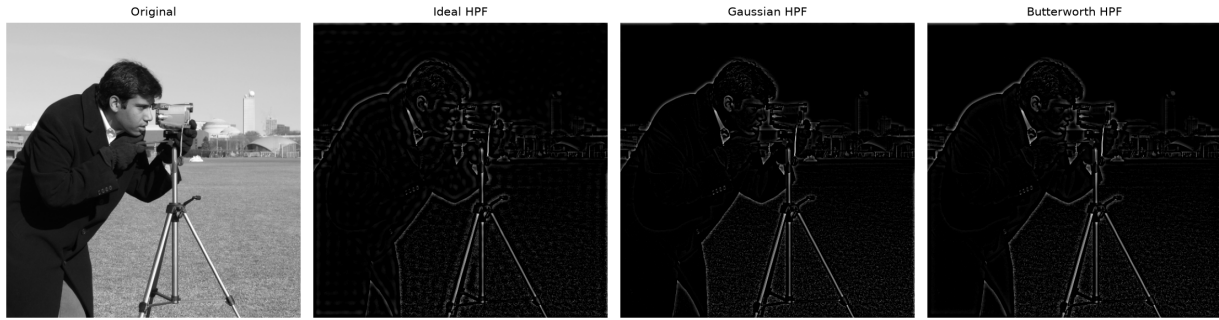
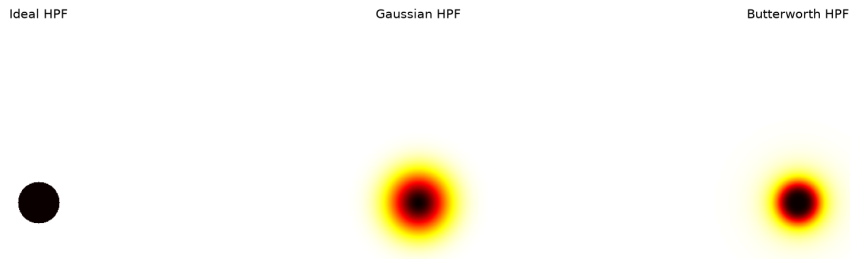
Figure 5: High-pass filter results for $D_0 = \sigma = 30$, $n = 2$.

Figure 6: Frequency responses of high-pass filters.

High-pass filters preserve edges, fine texture, and other rapidly-varying features – the content carried by high spatial frequencies – while suppressing smooth, slowly-varying regions (flat backgrounds, gradual shading, overall illumination), carried by low frequencies. Edges are abrupt intensity transitions, requiring high-frequency sinusoidal components to represent; removing the low frequencies has almost no effect on them. Smooth regions are made up almost entirely of low/near-DC frequency content, so zeroing that content removes them almost completely – exactly what Figure 4 shows: the cameraman’s silhouette, tripod, and grass texture remain visible while the smooth sky and ground are pushed to near-black.

Q3.2 – Ideal HPF vs. Gaussian HPF.

The **Gaussian HPF produces cleaner edges**, free of ringing, because $1 - H_{\text{gaussian},LPF}$ inherits the Gaussian’s smooth, discontinuity-free transition. The ideal HPF’s sharp cutoff introduces the same ringing artefact as Q2.2 (now visible as faint oscillatory halos around the extracted edges). A gradual transition (Gaussian) sacrifices precision in exactly which frequencies are removed, in exchange for artefact-free output; a sharp transition (ideal) is precise but rings.

Q3.3 – Effect on overall brightness; value of $H(0,0)$.

All three HPFs push the average image brightness toward zero, because $H(0,0) = 0$ **for every HPF type**, confirmed numerically in the notebook run:

$$\text{Ideal HPF } H(0,0) = 0.0 \quad \text{Gaussian HPF } H(0,0) = 0.0 \quad \text{Butterworth HPF } H(0,0) = 0.0$$

Since $F(0,0)$ is the sum of all pixel intensities (Q1.1), multiplying it by $H(0,0) = 0$ deletes the image’s average brightness before the inverse transform. The spatial-domain result therefore has a mean near zero; after clipping negative values to 0, the output looks very dark overall, with only edges/texture visibly brighter than the now-black background – exactly what Figure 4 shows.

Q3.4 – Butterworth order sweep, $n \in \{1, 2, 5, 10\}$ (fixed $D_0 = 30$).

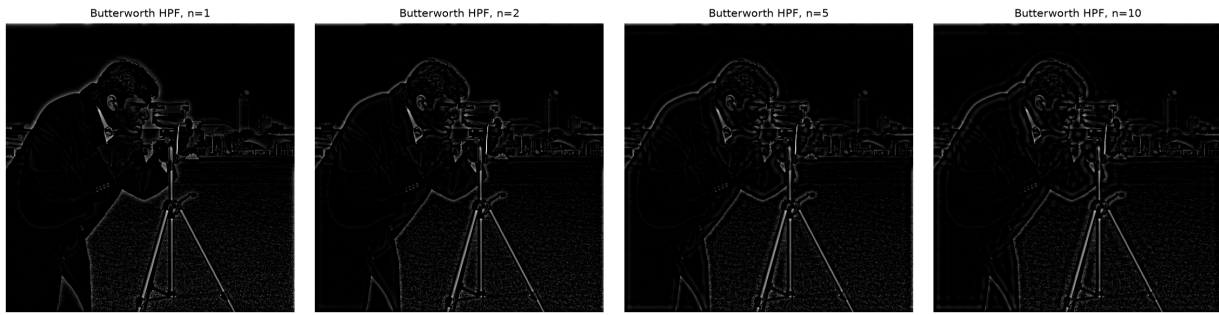


Figure 7: Butterworth HPF outputs for $n = 1, 2, 5, 10$ at fixed $D_0 = 30$.

As n increases from 1 to 10, the extracted edges become progressively thinner and higher-contrast, while faint ringing/haloing becomes increasingly visible around them – at $n = 1$ the output looks soft and smooth (closest to the Gaussian HPF), while at $n = 10$ it looks close to the ideal HPF, with crisp but slightly “double-edged” contours. This is the same trade-off as Q2.3: larger n makes Butterworth converge toward the ideal filter’s behaviour (sharper cutoff, more ringing/artefacts), while smaller n makes it converge toward the Gaussian’s behaviour (smoother, artefact-free, but less aggressive). In practice $n = 2-4$ is usually a reasonable compromise.

5 Task 4: High-Frequency Emphasis Filtering

5.1 Implementation

High-frequency emphasis combines an HPF with an offset to preserve low-frequency content:

$$H_{hfe}(u, v) = a + b \cdot H_{HPF}(u, v)$$

```
1 def hfe_filter(H_hpf, a, b):
2     return a + b * H_hpf
```

Listing 3: HFE implementation (Listing 3).

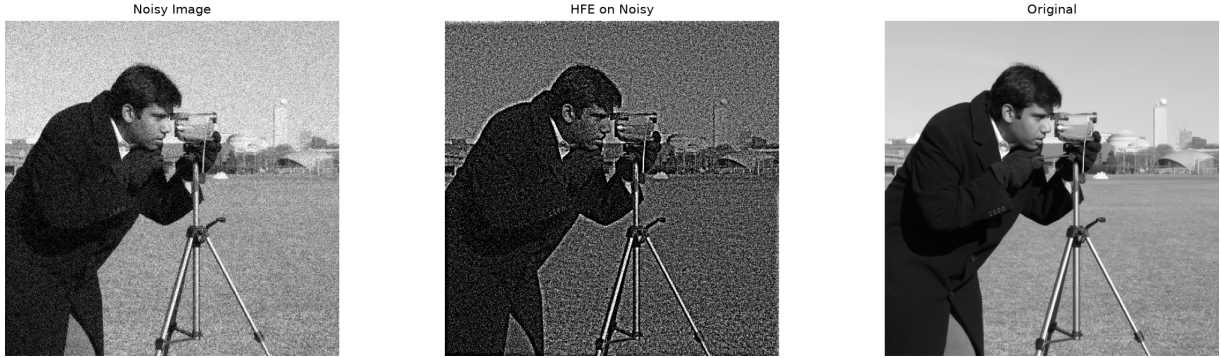
5.2 Results



Figure 8: HFE results for different (a, b) pairs (Butterworth HPF base, $D_0 = 30$, $n = 2$).

5.3 Noisy Image Results

(a, b)	$H_{hfe}(0, 0)$	Mean intensity (original = 0.506)	PSNR vs. original
(0.5, 1.0)	0.500	0.257	10.85 dB
(0.5, 1.5)	0.500	0.261	10.84 dB
(0.5, 2.0)	0.500	0.265	10.79 dB
(1.0, 2.0)	1.000	0.507	20.61 dB

Table 2: HFE DC gain, output brightness, and PSNR for each tested (a, b) pair.Figure 9: HFE ($a = 0.5$, $b = 1.5$) applied to a Gaussian-noise-corrupted image (var= 0.01).

5.4 Questions

Q4.1 – HFE vs. plain HPF brightness; role of parameter a .

A plain HPF has $H(0,0) = 0$ (Q3.3), so its output loses essentially all of its average brightness and looks very dark. HFE's constant offset a guarantees $H_{hfe}(0,0) = a + b \cdot H_{HPF}(0,0) = a - a$ non-zero DC gain whenever $a > 0$ – so a fraction a of the original image's brightness/low-frequency content passes straight through. The output therefore keeps a recognisable overall brightness instead of collapsing to near-black.

Q4.2 – Mathematical derivation of H_{hfe} 's DC component.

$$H_{hfe}(0,0) = a + b \cdot H_{HPF}(0,0) = a + b \cdot 0 = a$$

since $H_{HPF}(0,0) = 0$ for every HPF type used here. The DC gain of the combined HFE filter is exactly a , independent of b or of which HPF is used underneath – confirmed numerically in Table 2 ($H_{hfe}(0,0) = 0.5$ for all three $a = 0.5$ rows, and 1.0 for the $a = 1.0$ row). More generally, at any *low* (not just exactly-zero) frequency, $H_{HPF}(u,v) \approx 0$, so $H_{hfe}(u,v) \approx a$ there too – increasing a uniformly raises the floor of the frequency response across the entire low-frequency band, which is why larger a visibly retains more of the image's brightness and coarse structure.

Q4.3 – Best (a, b) pair.

The measured statistics in Table 2 make this a data-driven choice. All three $a = 0.5$ variants produce a similarly dark result (mean intensity ≈ 0.26 , about half the original's 0.506) and a similarly low PSNR (≈ 10.8 dB), regardless of b – confirming Q4.2's derivation that a alone sets the DC/low-frequency floor, while b only rescales the edge-emphasis term on top of it. $(a, b) = (1.0, 2.0)$ is the clear winner: it keeps the *same* strong edge boost ($b = 2.0$, identical to the most aggressive $a = 0.5$ case) while its mean intensity (0.507) and PSNR (20.61 dB, roughly double the $a = 0.5$ variants) are by far the closest to the original image. Raising a from 0.5 to 1.0 fixes the brightness problem

almost completely without giving up any edge emphasis, which b continues to control independently – exactly the two-knob design HFE is meant to provide.

Q4.4 – HFE applied to the noisy image.

Applying HFE to the Gaussian-noise-corrupted image clearly **amplifies the noise** rather than suppressing it – the HFE output (Figure 7, middle panel) looks visibly grainier than the noisy input itself (left panel). The underlying reason is spectral: additive Gaussian noise is approximately *broadband* – it has energy spread roughly uniformly across all spatial frequencies, including the high frequencies that $b \cdot H_{HPF}$ actively boosts. Since HFE cannot distinguish “true edge” high-frequency content from “noise” high-frequency content, boosting the former necessarily boosts the latter too – exactly why sharpening filters are normally applied *after* denoising, never before it, in a practical pipeline.

6 Task 5: Comparative Analysis

6.1 Summary Table

Filter Type	Best Use Case	Advantages	Disadvantages
Ideal LPF	Theoretical/pedagogical frequency-domain analysis	Precise, sharply-defined cutoff	Severe ringing (Gibbs artefacts); rarely used in practice
Gaussian LPF	General-purpose smoothing / photographic soft blur	No ringing; smooth, natural blur	No exact cutoff; harder to reason about detail loss
Butterworth LPF	Tunable denoising needing controllable transition	Order n trades ringing vs. sharpness	Extra parameter; still rings at high n
Ideal HPF	Theoretical edge/frequency analysis	Precise cutoff	Ringing; brightness collapses ($H(0,0)=0$)
Gaussian HPF	Artefact-free edge extraction	No ringing	Softer, less aggressive boost
Butterworth HPF	General sharpening / edge-detection pre-processing	Tunable order	DC always zeroed; ringing at high n
HFE	Sharpening that must preserve brightness	Preserves brightness via a ; tunable boost via b	Amplifies broadband noise

Table 3: Comparative analysis of frequency-domain filters, based on our experimental observations.

6.2 Filter Shape Comparison

The cross-sections make the ringing/order discussion visually concrete: the **ideal** filter is a perfect step (discontinuous derivative at D_0 , the source of ringing); the **Gaussian** curve is smooth everywhere with no discontinuity at any order of derivative; and the **Butterworth** curve sits between them, with its steepness directly controlled by n – visibly closer to the Gaussian’s smooth shoulder for small n and closer to the ideal step for large n .

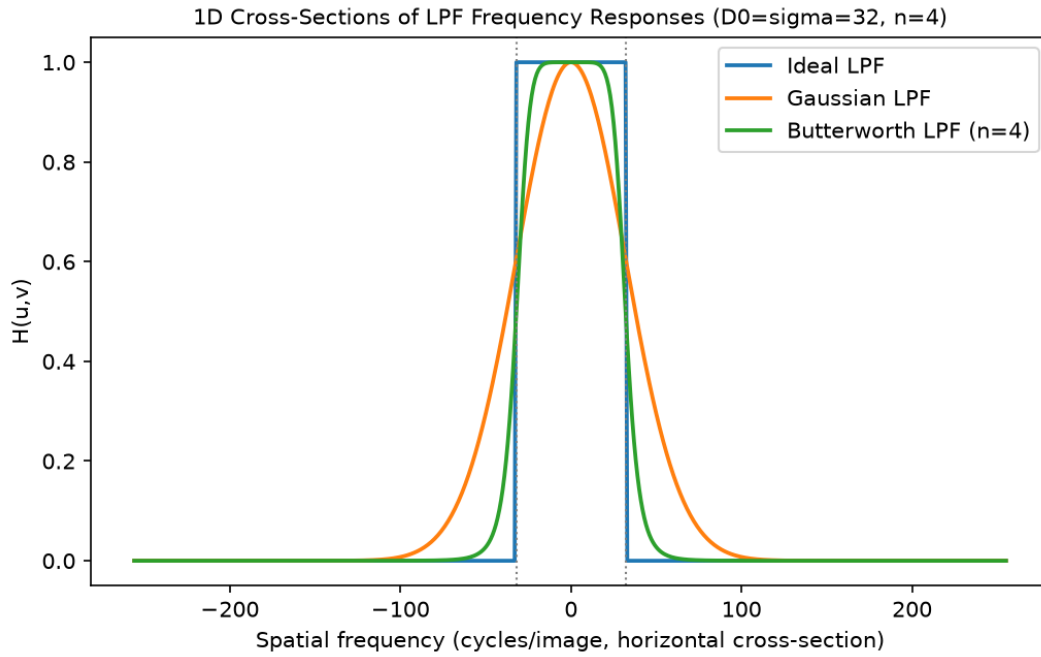


Figure 10: 1D cross-sections (middle row) of LPF frequency responses, $D_0 = \sigma = 32$, $n = 4$.

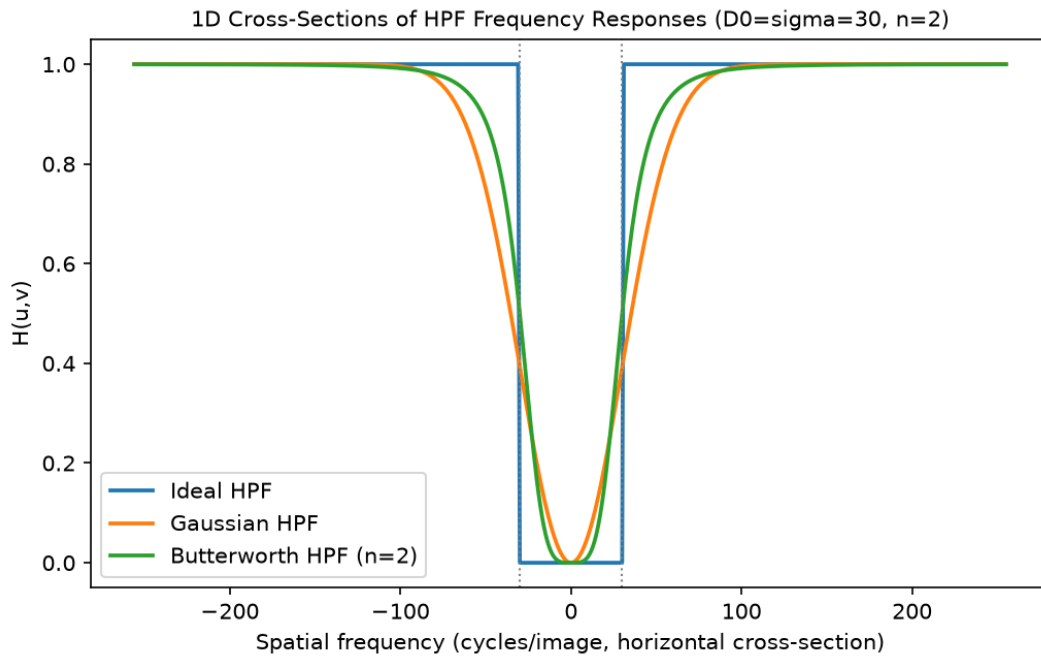


Figure 11: 1D cross-sections (middle row) of HPF frequency responses, $D_0 = \sigma = 30$, $n = 2$.

6.3 Questions

Q5.1 Summary Table: See the filled table in Section 6.1.

Q5.2 Filter Shape Comparison: See the two cross-section figures above. The shape of $H(u, v)$ near the cutoff directly predicts spatial-domain behaviour: a discontinuous step (ideal) produces a spatial kernel with infinite support and oscillating side-lobes $\Rightarrow$ ringing; a C^∞ -smooth curve with no discontinuity (Gaussian) produces a spatial kernel that decays smoothly with no oscillation $\Rightarrow$ no ringing but a less sharply-defined cutoff; the Butterworth curve's steepness (set by n) interpolates

continuously between these two extremes.

Q5.3 Real-World Applications:

(a) **Gaussian LPF** – consumer photography / portrait retouching (soft-focus, skin smoothing) and pre-processing before downsampling: it is the natural choice wherever a smooth, artefact-free blur is wanted and no exact cutoff frequency is required, since visible ringing would look like an obvious processing artefact.

(b) **Butterworth HPF** – medical imaging (enhancing bone/tissue boundaries in X-ray or MRI slices): the tunable order n lets a radiologist/engineer dial in just enough edge enhancement to make diagnostically relevant boundaries clearer, while keeping n low enough that ringing artefacts are not mistaken for genuine anatomical structure.

(c) **HFE filtering** – remote sensing / satellite imagery enhancement: analysts need crisper terrain/feature boundaries (roads, coastlines, field boundaries) for classification, but also need the image’s overall radiometric brightness preserved for further quantitative analysis; HFE’s tunable a is designed to satisfy both needs at once.

Q5.4 Convolution Theorem Connection:

The convolution theorem states that spatial-domain convolution corresponds exactly to frequency-domain multiplication:

$$g(x, y) = f(x, y) * h(x, y) \iff G(u, v) = F(u, v) \cdot H(u, v)$$

Every filter applied in this report via `apply_filter` (multiplying $H(u, v)$ onto $F(u, v)$ and inverse-transforming) is therefore mathematically identical to convolving the original image with $h(x, y) = \mathcal{F}^{-1}\{H(u, v)\}$ in the spatial domain. This equivalence matters for two practical reasons: **computational efficiency**, since direct spatial convolution with a large kernel costs $O(MNK^2)$ for a $K \times K$ kernel, while the FFT-based route costs only $O(MN \log(MN))$ regardless of the spatial kernel’s effective size; and **design flexibility**, since it is often far more intuitive to directly specify a filter’s desired frequency response than to hand-derive an equivalent spatial convolution kernel – the theorem guarantees that whatever frequency response we design can always be realised as some spatial-domain filter, even if we never explicitly compute what that spatial kernel looks like.

Q5.5 Student Reflections:

Muhammad Umer (F2023266912): The trickiest part of this activity for me was getting comfortable with `fftshift` and frequency-domain indexing – my first attempt at the distance matrix $D(u, v)$ didn’t use `np.fft.fftfreq`, and the resulting filters looked completely wrong (blurring the corners instead of the centre) because I hadn’t accounted for how NumPy’s raw FFT output wraps high frequencies into the middle of the array. Once I traced through `distance_matrix` step by step and printed D before and after `fftshift`, it became clear that the filter itself must be built in the same (unshifted) coordinate system as `np.fft.fft2`’s raw output, and `fftshift` should only be applied when *displaying* a spectrum or filter, never before multiplying it onto F . That distinction – display-space vs. computation-space shifting – was the key insight that resolved every downstream bug in Tasks 2–4. (108 words)

Haseeb Ullah (F20232661009): My main challenge was understanding why the Butterworth HPF code special-cases $D = 0$ with a boolean mask instead of just writing the formula directly like the LPF version does. Running the naive version threw a `RuntimeWarning: divide by zero` because the formula uses $D_0/D(u, v)$, undefined at the origin. I resolved this by working out on paper what $H_{HPF}(0, 0)$ *should* be (the limit of a high-pass filter at DC is always zero, matching the “1 minus low-pass” definition used for the ideal/Gaussian filters), and then verified the masked implementation gives exactly `0.0` there. This also directly explained the surprisingly dark HPF output images in Task 3, tying the coding bug I’d hit to the actual DIP theory in Q3.3. (114 words)

7 Conclusion

This activity provided hands-on experience with the full frequency-domain filtering pipeline: computing and interpreting the 2D DFT and its log-magnitude spectrum; designing and applying ideal, Gaussian, and Butterworth low-pass and high-pass filters; quantifying filtering quality with PSNR; combining high-pass filtering with a DC-preserving offset via high-frequency emphasis; and analysing all of the above both visually and through 1D frequency-response cross-sections.

The consistent thread across every task is the **trade-off between sharpness of the frequency-domain cutoff and the presence of spatial-domain ringing artefacts**: the ideal filter's discontinuous step gives the most precise (but artefact-prone) cutoff; the Gaussian filter's smooth response gives the least precise (but artefact-free) cutoff; and the Butterworth filter's order parameter n provides a single, continuously tunable knob between these two extremes – a pattern that recurs identically whether filtering low frequencies out (Task 2), high frequencies out (Task 3), or blending the two via HFE (Task 4). Understanding this trade-off, together with the convolution theorem's guarantee that every frequency-domain design choice corresponds to some equivalent spatial-domain kernel, is the core practical takeaway of frequency-domain image filtering.

References

1. Gonzalez, R.C. and Woods, R.E., *Digital Image Processing*, 4th ed., Pearson, 2018.
2. Szeliski, R., *Computer Vision: Algorithms and Applications*, Springer, 2010.
3. Pratt, W.K., *Digital Image Processing*, 4th ed., Wiley, 2007.
4. Burger, W. and Burge, M.J., *Digital Image Processing: An Algorithmic Introduction*, Springer, 2016.
5. NumPy Documentation, <https://numpy.org/doc/>
6. scikit-image Documentation, <https://scikit-image.org/docs/>

A Complete Code Listing

The full, executable source is in `cas/ca_06/ca_06.ipynb`; the key routines are reproduced here for reference.

```

1 def compute_fft(image):
2     F = np.fft.fft2(image)
3     F_shift = np.fft.fftshift(F)
4     magnitude = np.log1p(np.abs(F_shift))
5     return F, F_shift, magnitude
6
7 def distance_matrix(M, N):
8     u = np.fft.fftfreq(M) * M
9     v = np.fft.fftfreq(N) * N
10    V, U = np.meshgrid(v, u)
11    return np.sqrt(U**2 + V**2)
12
13 def apply_filter(image, H):
14     F = np.fft.fft2(image)
15     G = H * F
16     g = np.real(np.fft.ifft2(G))
17     return np.clip(g, 0, 1)
18
19 def ideal_lpf(D, D0):
20     return (D <= D0).astype(float)
21
22 def gaussian_lpf(D, sigma):
23     return np.exp(-(D**2) / (2 * sigma**2))
24
25 def butterworth_lpf(D, D0, n):
26     return 1 / (1 + (D / D0)**(2 * n))
27
28 def ideal_hpf(D, D0):
29     return 1 - ideal_lpf(D, D0)
30
31 def gaussian_hpf(D, sigma):
32     return 1 - gaussian_lpf(D, sigma)
33
34 def butterworth_hpf(D, D0, n):
35     H = np.zeros_like(D)
36     mask = D > 0
37     H[mask] = 1 / (1 + (D0 / D[mask])** (2 * n))
38     return H
39
40 def hfe_filter(H_hpf, a, b):
41     return a + b * H_hpf

```

Listing 4: Shared setup: FFT helpers, distance matrix, and all filter functions.

```

1 image = img_as_float(data.camera())
2 M, N = image.shape
3 D = distance_matrix(M, N)
4
5 cutoffs = [8, 16, 32, 64]
6 n_order = 4
7 results_ideal, results_gaussian, results_butter = [], [], []
8 for D0 in cutoffs:
9     results_ideal.append(apply_filter(image, ideal_lpf(D, D0)))
10    results_gaussian.append(apply_filter(image, gaussian_lpf(D, D0)))
11    results_butter.append(apply_filter(image, butterworth_lpf(D, D0, n_order)))
12
13 for i, D0 in enumerate(cutoffs):

```

```
14 p_ideal = psnr(image, results_ideal[i], data_range=1.0)
15 p_gauss = psnr(image, results_gaussian[i], data_range=1.0)
16 p_butter = psnr(image, results_butter[i], data_range=1.0)
17 print(f"D0={D0:2d} | Ideal:{p_ideal:6.2f} dB | Gaussian:{p_gauss:6.2f} dB |
      Butterworth:{p_butter:6.2f} dB")
```

Listing 5: Task 2 driver: low-pass filter sweep and PSNR computation.

```
1 H_butt_hp_base = butterworth_hpf(D, D0=30, n=2)
2 params = [(0.5, 1.0), (0.5, 1.5), (0.5, 2.0), (1.0, 2.0)]
3 hfe_results = [apply_filter(image, hfe_filter(H_butt_hp_base, a, b)) for a, b in
4               params]
5 noisy = random_noise(image, mode='gaussian', var=0.01)
6 H_hfe_best = hfe_filter(H_butt_hp_base, a=0.5, b=1.5)
7 noisy_hfe = apply_filter(noisy, H_hfe_best)
```

Listing 6: Task 4 driver: HFE parameter sweep and the noisy-image experiment.